COMPUTER ENGINEERING

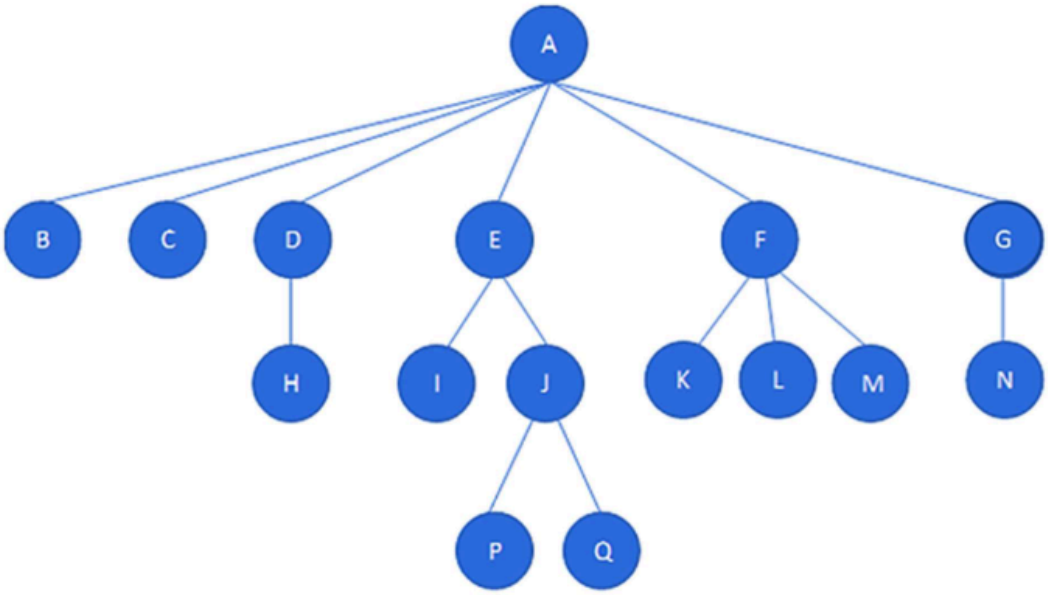
QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S1
DATE PERFORMED:	09/21/26	**DATE SUBMITTED:**	09/24/26
NAME:	Medina, Jomz Ryle E.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 9.1 **Tree ADT**

1. PROCEDURE OUTPUT

ILO A: Create C++ code to implement a general tree, binary tree and binary search tree.



Task 1: Create code in C++ that will create a tree as shown in the figure above. Use linked lists as the internal representation of this tree. **Indicate your code screenshot and comments in table 9-1.**

Task 2: Complete the following table:

Node	Height	Depth
A		
B		
C		
D		
E		
F		
G		
H		
I		
J		
K		
L		
M		
N		
P		
Q		

Include this completed table as table 9-2 in section 6.

ILO B: Create C++ code for implementation of tree traversal methods such as pre-order, in-order and post-order traversal.

Task 3: After implementing the code for above, answer the following:

3.1 Given the tree diagram, find the result of the pre-order, post-order and in-order traversal strategies by hand. **Include this as table 9-3 in section 6.**

Pre-order	
Post-order	
In-order	

3.2 Create a function for pre-order, post-order and in-order traversal. Make sure that each function displays an output into the console. Once you have the output, **create and fill table 9-4 in section 6** so that it contains the screenshot of the function, screenshot of the output and your observations. Your observations consist of a comparison between output in #1 and the output of your functions in #2.

3.3. Create a function called `findData` that will take the following parameter: `"CHOICE, KEY"`. `Choice` will determine which traversal method will be used to traverse the tree to find the second parameter `KEY`. If the `KEY` is found, the value will be displayed such that: "{KEY} was found!". Otherwise, no output. **Include your output as table 9-5 in section 6.**

3.4. Add a new leaf node called which will contain the data value of **"O"**. Make this node the child of nodeG. Perform the **findData** function you created in #3, what can you observe about the output? **Include your output and answers as table 9-6 in section 6.**

Code Screenshot (generalTree.h):

```
#ifndef GENERALTREE_H
#define GENERALTREE_H
#include <iostream>

template <typename T>
class GeneralTree{

    private:
        // Key Creation
        T key;
        // Left and Right Pointers
        GeneralTree<T>* left;
        GeneralTree<T>* right;
    public:
        // Constructor
        GeneralTree(T value);
        GeneralTree<T>* insert(T value);
        void display();
};

// CONSTRUCTOR
template <typename T>
GeneralTree<T>::GeneralTree(T value){
    key = value;
    left = nullptr;
    right = nullptr;
}
```

```cpp
// ADD CHILD AT END
template <typename T>
GeneralTree<T>* GeneralTree<T>::insert(T value){
    GeneralTree<T>* newNode = new GeneralTree<T>(value);

    if(left == nullptr){
        left = newNode;
    } else {
        GeneralTree<T>* current = left;
        while(current->right != nullptr){
            current = current->right;
        }
        current->right = newNode;
    }

    return newNode;
}

// DISPLAY FUNCTIONS
template <typename T>
void GeneralTree<T>::display(){
    if(left != nullptr){
        std::cout << key << ": ";
        GeneralTree<T>* current = left;

        while(current != nullptr){
            std::cout << current->key << " ";
            current = current->right;
        }
        std::cout << "\n";

        current = left;
        while(current != nullptr){
            current->display();
            current = current->right;
        }
    }
}

#endif
```

```
A: B C D E F G
D: H
E: I J
J: P Q
F: K L M
G: N
```

(Terminal)

Observations/Analysis:

The code above basically creates a general tree with a linked-list type implementation where each of the nodes can store a letter through ‘key’ and have pointers to the left and right child. The **insert()** function adds children in order which allows a parent to have two or more children unlike with the binary search tree that we had created in our lecture. Additionally, I added a display function to really show that the implementation works.

Node	Height	Depth
A	3	0
B	0	1
C	0	1
D	1	1
E	2	1
F	1	1
G	1	1
H	0	2
I	0	2
J	1	2
K	0	2
L	0	2
M	0	2
N	0	2
P	0	3
Q	0	3

ILO A Table 9-2

Pre-Order	A B C D H E I J P Q F K L M G N
Post-Order	B C H D I P Q J E K L M F N G A
In-Order	B A C H D I E P J Q K F L M N G

ILO B (Manual Calculations) Table 9-3

preOrder() Code Output:

```
template <typename T>
void GeneralTree<T>::preOrder(){
    std::cout << key << " ";
    GeneralTree<T>* current = left;
    while(current != nullptr){
        current->preOrder();
        current = current->right;
    }
}
```

inOrder() Code Output:

```
template <typename T>
void GeneralTree<T>::inOrder(){
    GeneralTree<T>* current = left;
    if(current != nullptr){
        current->inOrder();
        current = current->right;
    }

    std::cout << key << " ";
    while(current != nullptr){
        current->inOrder();
        current = current->right;
    }
}
```

postOrder() Code Output:

```
template <typename T>
void GeneralTree<T>::postOrder(){
    GeneralTree<T>* current = left;
    while(current != nullptr){
        current->postOrder();
        current = current->right;
    }
    std::cout << key << " ";
}
```

Terminal Output:

```
Pre-Order:
A B C D H E I J P Q F K L M G N

Post-Order:
B C H D I P Q J E K L M F N G A

In-Order:
B A C H D I E P J Q K F L M N G
```

Observations/Analysis:

The following code above applies the same logic that we already created with the initial BinarySearchTree library file, however, tweaked to also be applicable for general trees. Upon comparison, it successfully matches with the manually traced table from earlier and all three methods visit the node with order different per function.

ILO B Table 9-4

findData() (generalTree.h) Code Output:

```
template <typename T>
void GeneralTree<T>::findData(int choice, T searchKey){
    if(choice < 1 || choice > 3){
        return;
    }

    if(choice == 1 && key == searchKey){
        std::cout << key << " was found!\n";
    }

    GeneralTree<T>* current = left;

    if(choice == 2){
        if(current != nullptr){
            current->findData(choice, searchKey);
            current = current->right;
        }
        if(key == searchKey){
            std::cout << key << " was found!\n";
        }
    }

    while(current != nullptr){
        current->findData(choice, searchKey);
        current = current->right;
    }

    if(choice == 3 && key == searchKey){
        std::cout << key << " was found!\n";
    }
}
```

```
Finding I using pre-order:
I was found!
Finding I using in-order:
I was found!
Finding I using post-order:
I was found!

Finding O using pre-order:
Finding O using in-order:
Finding O using post-order:
```

(Terminal)

Observations/Analysis:
The function, findData() checks for each of the nodes and compares if the key is absent where it will print nothing. If it does manage to find the key value, it prints that said value was found.

ILO B (using findData() function) Table 9-5

Inserting ‘O’ under node G (main.cpp):

```
// INSERT
std::cout<<"\n";
std::cout<<"Inserting 'O'\n";
nodeG->insert('O');
std::cout << "Pre-order search: ";
tree.findData(1, 'O');
std::cout << "In-order search: ";
tree.findData(2, 'O');
std::cout << "Post-order search: ";
tree.findData(3, 'O');
```

```
Inserting 'O'
Pre-order search: O was found!
In-order search: O was found!
Post-order search: O was found!
```

(Terminal)

Observations/Analysis:
Before ‘O’ was added via insertion, pre, post, and in order produced nothing. However, after adding it via appending O to the tree, it now shows up as found meaning that the code runs successfully.

ILO B (Insertion) Table 9-6

2. SUPPLEMENTARY OUTPUT

ILO C: Solve given problems using the tree data structure's implementation in C++

Step 1: Implement a binary search tree that will take the following values: 2, 3, 9, 18, 0, 1, 4, 5.

(Screenshot of code)

Step 2: Create a diagram to show the tree after all values have been inserted. Then, with the use of visual aids (like arrows and numbers) indicate the traversal order for in-order, pre-order and post-order traversal on the diagram.

(Screenshot of tree diagram)

(Screenshot of tree diagram with indicated in-order traversal)

(Screenshot of tree diagram with indicated pre-order traversal)

(Screenshot of tree diagram with indicated post-order traversal)

Step 3: Compare the different traversal methods. In-order traversal was performed with what function?

(Screenshot output needed)

Given the same input values above, what is the output with different traversal methods? For each output below, indicate your observation: is the output different from the pre-order and post-order traversal that you indicated in the diagrams shown in item #2.

Pre-order Traversal

(Screenshot created function)

(Screenshot console output)

Post-order Traversal

(Screenshot created function)

(Screenshot console output)

Step 1:

(BST.h)

main.cpp ● | BST.h ✕ | generalTree.h

BST.h > preOrder()

```h
#ifndef BST_H
#define BST_H
#include <iostream>

template <typename T>
class BinarySearchTree{

    private:
        // Key Creation
        T key;
        // Left and Right Pointers
        BinarySearchTree<T>* left;
        BinarySearchTree<T>* right;
    public:
        // Constructor
        BinarySearchTree(T value);
        // Insert left and right
        void insert(T value);
        // Traversal
        void preOrder();
        void inOrder();
        void postOrder();
        // Searching
        void findData();
};

// Constructor
template <typename T>
BinarySearchTree<T>::BinarySearchTree(T value){
    key = value;
    left = nullptr;
    right = nullptr;
}

template <typename T>
void BinarySearchTree<T>::insert(T value){

    // Whether to insert left
    if(value < key){
        if(left == nullptr){
            left = new BinarySearchTree<T>(value);
        } else {
            left->insert(value);
        }
    }

    // Whether to insert right
    else if(value > key){
        if(right == nullptr){
            right = new BinarySearchTree<T>(value);
        } else {
            right->insert(value);
        }
    }

}
```

```h
// TRAVERSAL
template <typename T>
void BinarySearchTree<T>::preOrder(){
    std::cout << key << " ";
    if(left != nullptr){
        left->preOrder();
    }

    if(right != nullptr){
        right->preOrder();
    }
}

template <typename T>
void BinarySearchTree<T>::inOrder(){
    if(left != nullptr){
        left->inOrder();
    }
    std::cout<<key<<" ";
    if(right != nullptr){
        right->inOrder();
    }
}

template <typename T>
void BinarySearchTree<T>::postOrder(){
    if(left != nullptr){
        left->postOrder();
    }
    if(right != nullptr){
        right->postOrder();
    }
    std::cout<<key<<" ";
}

// SEARCHING
template <typename T>
void BinarySearchTree<T>::findData(){

}

#endif
```

(main.cpp & terminal output):

```cpp
    // SUPPLEMENTARY
    int values1[] = {3, 9, 18, 0, 1, 4, 5};
    int size = sizeof(values1)/sizeof(values1[0]);
    BinarySearchTree<int> tree2(2);

    for (int i = 0; i < size; ++i){
    tree2.insert(values1[i]);
    }


    std::cout<<"Binary Search Tree (2, 3, 9, 18, 0, 1, 4, 5):\n\n";

    std::cout<<"Pre-Order:\n";
    tree2.preOrder();
    std::cout<<"\n\n";

    std::cout<<"In-Order:\n";
    tree2.inOrder();
    std::cout<<"\n\n";

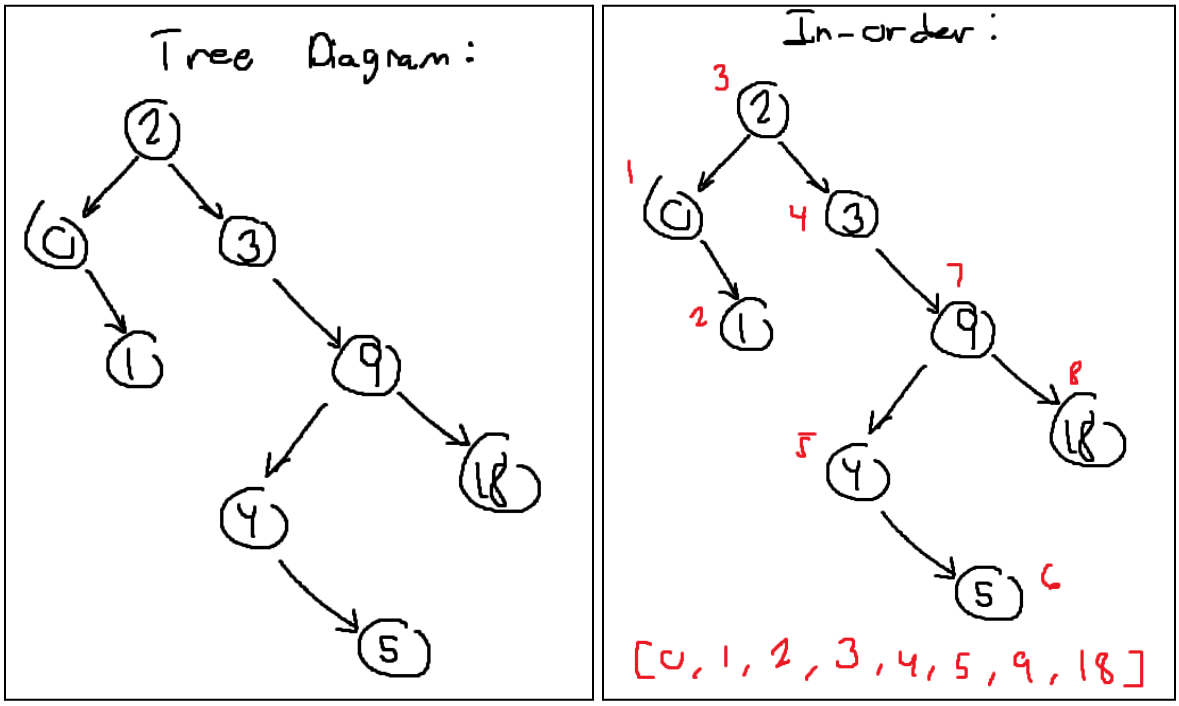
    std::cout<<"Post-Order:\n";
    tree2.postOrder();
    return 0;
}
```

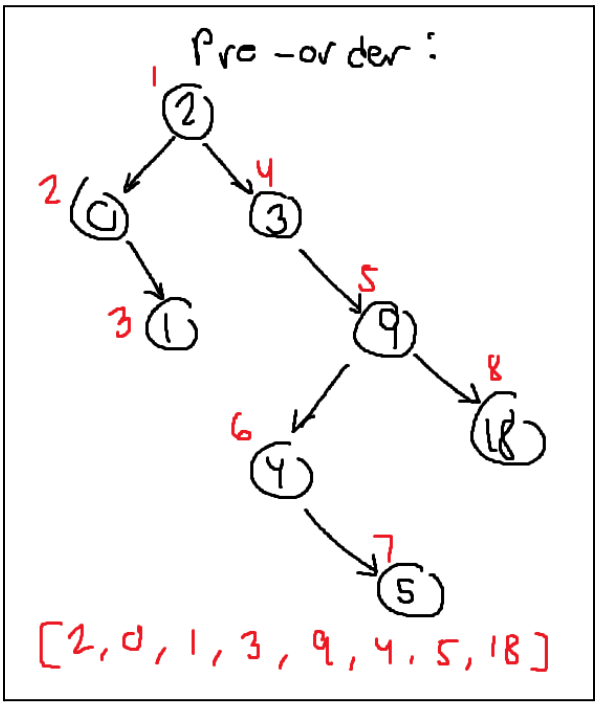
```
Binary Search Tree (2, 3, 9, 18, 0, 1, 4, 5):
Pre-Order:
2 0 1 3 9 4 5 18

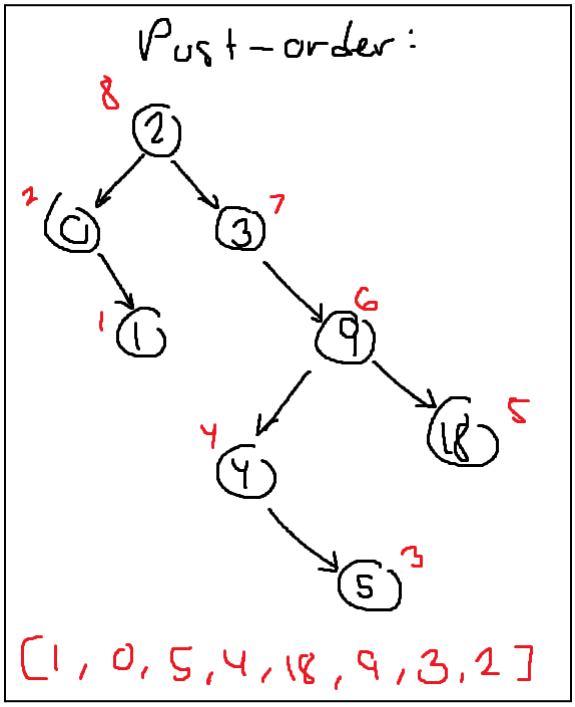
In-Order:
0 1 2 3 4 5 9 18

Post-Order:
1 0 5 4 18 9 3 2
```

Step 2:







Step 3:

"In-order traversal was performed with what function?"

The In-order traversal was performed by the member function, '**inOrder()**' which was called via '**tree2.inOrder()**' where the logic follows the visiting of the left subtree, displaying the node we're currently at, then going towards the right subtree (left, root, right).

Created Function:

```
71  template <typename T>
72  void BinarySearchTree<T>::inOrder(){
73      if(left != nullptr){
74          left->inOrder();
75      }
76      std::cout<<key<<" ";
77      if(right != nullptr){
78          right->inOrder();
79      }
80  }
```

```
93      std::cout<<"In-Order:\n";
94      tree2.inOrder();
95      std::cout<<"\n\n";
```

Console Output:

```
In-Order:
0 1 2 3 4 5 9 18
```

Pre-order Traversal:

Created Function:

```
58  // TRAVERSAL
59  template <typename T>
60  void BinarySearchTree<T>::preOrder(){
61      std::cout << key << " ";
62      if(left != nullptr){
63          left->preOrder();
64      }
65
66      if(right != nullptr){
67          right->preOrder();
68      }
69  }
```

```
89      std::cout<<"Pre-Order:\n";
90      tree2.preOrder();
91      std::cout<<"\n\n";
```

Console Output:

```
Pre-Order:
2 0 1 3 9 4 5 18
```

Observation:

In pre-order traversal, the output differs as it follows the logic of displaying the parent before going to the left then right subtrees, hence, output is **[2, 0, 1, 3, 9, 4, 5, 18]** (root, left, right).

Post-order Traversal:

Created Function:

```
82  template <typename T>
83  void BinarySearchTree<T>::postOrder(){
84      if(left != nullptr){
85          left->postOrder();
86      }
87      if(right != nullptr){
88          right->postOrder();
89      }
90      std::cout<<key<<" ";
91  }
```

```
97      std::cout<<"Post-Order:\n";
98      tree2.postOrder();
99      return 0;
```

Console Output:

```
Post-Order:
1 0 5 4 18 9 3 2
```

Observation:

In post-order, the output is different once more as the parent node is now displayed last where left is first, followed by the right subtree, producing the output, **[1, 0, 5, 4, 18, 9, 3, 2]** (left, right, root).

Conclusion:

In this activity, I learned a lot about how to actually apply the concepts and relationships of in, pre, and post-order configurations between their parent and child nodes as well as how binary and general search trees are implemented through code. The general search tree design that I decided upon was basically just an implementation of linked lists. Additionally, I also went over height and depth and definitely have a more in-depth (pun not intended) grasp of what they actually mean with how they were practically applied in the activity where I had to personally research and make sure that what I was answering was correct. Overall, this lab activity has contributed to my understanding of tree structures as well as how different traversals work within them and how they implement recursive calling, pointers, as well as linked nodes.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **None**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **N/A**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **N/A**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.